SCHOOL OF TECHNOLOGY

WOXSEN UNIVERSITY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Python

PROJECT TITLE

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR ONLINE COURSE LEARNING PROGRESS MANAGEMENT SYSTEM

Program	B.Tech Computer Science and Engineering (Specialisation in Artificial Intelligence and Machine Learning)
Section	AI-ML LEOPARDS
Faculty	Dr. C Nagaraju
Date of Submission	12-10-2026
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0102216	**PriyanshuVerma**	9971420821	priyanshu.verma_2029@woxsen.edu.in

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the Database Management Systems (DBMS) course.

I thank my course faculty,Dr. C.Nagaraju , for the guidance, suggestions, and support throughout the design and development of this project. The guidance helped me understand the practical application of database concepts such as ER modelling, relational schema design, normalization, SQL, constraints, queries, and database application development.

This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem.

UNDERTAKING

I hereby declare that the project entitled "DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR ONLINE COURSE LEARNING PROGRESS MANAGEMENT SYSTEM", submitted as part of the Project-Based Learning activity for the Database Management Systems (DBMS) course, is my individual work carried out under the guidance of the course faculty.

I undertake that the project report, database design, SQL scripts, application source code, results, and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets, or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Signature: _____

Date : 12-10-2026

Place : Kamkole, Hyderabad

Name : Priyanshu Verma

Roll Number : 25WU0102216

Program : B.Tech CSE (AI & ML)

TABLE OF CONTENTS

Contents	Page No.
Abstract	v
LIST OF TABLES	vi
LIST OF FIGURES	vii
LIST OF ACRONYMS	viii
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	**1**
1.1 Problem Statement	1
1.2 Objectives	2
1.3 Scope and Assumptions	2
1.4 Proposed Solution Strategy	3
1.5 Functional Requirements	4
1.6 Entity-Relationship Diagram (ERD)	5
1.7 Entity and Relationship Summary	6
2. LOGICAL DATABASE DESIGN	**8**
2.1 Relational Schema	8
2.2 Keys and Integrity Constraints	9
2.3 Functional Dependencies	11
2.4 Normalization up to Third Normal Form (3NF)	12

2.5 Data Dictionary 14

2.6 Implementation of Logical Design: DDL and Sample Data 16

3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN 18

3.1 Application Overview 18

3.2 Development Environment and Database Connectivity 18

3.3 Application Modules and Database Mapping 19

3.4 CRUD Operations 20

3.5 Business Rules, Validation and Error Handling 21

3.6 Reports and Database Operations 22

3.7 Application Screens 23

3.8 Testing and Results 26

4. CONCLUSION 28

4.1 Limitations and Future Enhancements 28

5. REFERENCES 29

6. APPENDICES 31

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	2
Table 2	Proposed Solution Strategy	3
Table 3	Functional Requirements	4
Table 4	Entity and Relationship Summary	6
Table 5	Relational Schema	8
Table 6	Keys and Integrity Constraints	9
Table 7	Normalization Status up to 3NF	13
Table 8	Data Dictionary	14
Table 9	Database Implementation Environment	16
Table 10	Application Development Environment	18
Table 11	Application Module-to-Database Mapping	19
Table 12	CRUD Operations	20
Table 13	Business Rules, Validation and Error Handling	21
Table 14	Reports and Database Operations	22
Table 15	Test Cases	26

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Proposed System	5
Figure 2	Main Application Dashboard and Navigation Interface	23
Figure 3	Learner Management and Registration Interface (CRUD Create/Read)	24
Figure 4	Course Enrolment Desk and Status Manager (CRUD Update)	24
Figure 5	Lesson Progress Tracker and Quiz Assessment Evaluation Interface	25
Figure 6	Comprehensive Learner Progress Analytics and Reporting Screen	25

LIST OF ACRONYMS

Acronym	Expansion
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
AI	Artificial Intelligence
BCNF	Boyce-Codd Normal Form
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
ER	Entity-Relationship
ERD	Entity-Relationship Diagram
FK	Foreign Key
GUI	Graphical User Interface
IDE	Integrated Development Environment
ML	Machine Learning
PBL	Project-Based Learning
PK	Primary Key
RDBMS	Relational Database Management System
SQL	Structured Query Language
UI	User Interface
UQ	Unique Constraint

ABSTRACT

The rapid expansion of web-based educational environments demands rigorous database architectures capable of tracking student navigation, modular completion states, and formative assessment scores with guaranteed transactional integrity. Traditional flat file and spreadsheet implementations suffer from update anomalies, unconstrained duplicate registrations, and computational latency when deriving real-time progress metrics. This project presents the design and implementation of an end-to-end relational database management system for an Online Course Learning Progress Management System, titled learning_progress, deployed on MySQL 8.0 and interfaced through a modular Python Tkinter graphical user interface. Conceptual data modelling derives ten strongly normalized relations strictly satisfying Third Normal Form and Boyce-Codd Normal Form, eliminating data redundancy while enforcing domain check constraints and cascading referential integrity. The desktop client utilizes mysql-connector-python to perform parameterized CRUD operations, manage course enrolments, record timestamped lesson progress percentages, administer multi-attempt quizzes, and generate real-time aggregated academic progress reports via SQL multi-table joins and analytical group functions. The completed database application demonstrates flawless constraint enforcement, eliminates orphaned progress records, and provides educational mentors with instantaneous visibility into learner completion rates.

KEYWORDS

Keywords: Relational Database, ER Modeling, Database Normalization, SQL Constraints, Python Tkinter, Online Course Learning Progress

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

In modern higher education institutions and e-learning ecosystems, students increasingly engage with digital learning platforms hosting modular, multi-topic curricula. While students interact with varied educational assets—including video lectures, foundational reading materials, interactive exercises, and evaluative quizzes—academic coordinators and instructors face severe difficulties tracking granular learning pacing and progress. Conventional record-keeping systems commonly rely on flat spreadsheets, ad-hoc file storage, or disconnected application logs. Such manual and semi-automated workflows suffer from critical data integrity issues, including duplicate course enrolments, conflicting records of lesson completion states, lack of referential constraints between modules and courses, and the complete inability to track historical assessment attempts over time.

1.2 Objectives

The core aim of this Project-Based Learning (PBL) activity is to engineer and evaluate an end-to-end database solution that manages course structures, learner registrations, lesson tracking, and assessment performance. The measurable project objectives are formulated as follows:

- **Design** a robust, ten-table relational database schema conforming strictly to Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF), enforcing primary keys, foreign keys, unique alternate keys, domain check constraints, and cascading referential integrity actions.
- **Develop** a modular, multi-tabbed desktop Graphical User Interface (GUI) in Python using Tkinter and ttk, interfacing with MySQL via mysql-connector-python to execute secure, parameterized SQL transactions for administrative and academic tasks.

• **Implement** granular lesson-level tracking mechanisms that record timestamps, access states (Not Started, In Progress, Completed), and exact progress percentages, preventing duplicate tracking via compound unique constraints.

• **Validate** formative assessment integrity by recording sequential quiz attempts, automatically scoring submissions against passing benchmarks, logging pass/fail verdicts, and preventing fraudulent score submissions.

• **Generate** real-time analytical reports through multi-table SQL joins, aggregate expressions, and conditional grouping to deliver actionable insights into individual course completion rates and overall academic engagement.

1.3 Scope and Assumptions

The project defines clear boundaries between functional capabilities implemented within the relational schema and peripheral enterprise infrastructure excluded from the prototype. Table 1 presents the precise scope, exclusions, and operational assumptions governing the system.

Table 1. Scope and Assumptions

Item	Description
In Scope	• Relational database modelling of 10 tables: LEARNER, INSTRUCTOR, COURSE, ENROLMENT, MODULE, LESSON, LESSON_PROGRESS, QUIZ, QUESTION, and ATTEMPT. • Implementation of candidate keys, primary keys, foreign keys with referential actions (CASCADE / RESTRICT), NOT NULL, UNIQUE, and CHECK constraints.
Out of Scope / Exclusions	• Payment gateway integration, course fee invoicing, and financial transaction processing. • Binary video streaming storage and content hosting (video URLs and durations are tracked metadata). • Web-based multi-tenant authentication protocols (OAuth2/SSO/JWT) and cloud identity federation. • Automated biometric exam proctoring and computer-vision-based cheating detection.

Assumptions	• All learners and instructors possess valid institutional email addresses adhering to standard domain formats. • Course curricula follow strict hierarchical decomposition: Course → Modules → Lessons/Quizzes. • The MySQL 8.0 server is running locally with InnoDB as the default transactional storage engine. • The desktop application executes on standard Windows workstations running Python 3.10+.

1.4 Proposed Solution Strategy

The proposed solution addresses the identified challenges by adopting a disciplined relational database architecture that unifies curricular hierarchies and student participation logs. Table 2 details the solution strategy across primary stakeholder roles, major functional modules, end-to-end user workflows, business rules, and anticipated reporting outputs.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	• Academic Administrator: Registers learners, manages course-instructor allocations, and monitors platform-wide completion reports. • Course Instructor: Oversees course curricula, inspects student pacing, and reviews quiz attempts. • Enrolled Learner: Traverses curricular modules, updates lesson completion states, and submits quiz assessments.
Major Application Modules	1. Dashboard: Displays platform metrics (total learners, active courses, enrolments, quiz attempts). 2. Learner Management: Learner onboarding, profile maintenance, status controls, and search filters. 3. Course Catalog: Curricular navigation displaying courses, modules, lessons, and assigned instructors. 4. Enrolment Desk: Registration of learners into courses with lifecycle status tracking. 5. Lesson Progress Tracker: Granular slider-based completion tracking and timestamp recording. 6. Quizzes & Assessment: Multi-attempt quiz administration,

	dynamic scoring, and pass/fail evaluation. 7. Analytical Reporting: Multi-table SQL aggregation queries generating real-time progress matrices.
Proposed Workflow	Administrator/Learner registers student profile → Learner enrols into target course → Learner navigates modules and updates lesson progress percentage → Learner undertakes module quiz → System calculates score and records timestamped attempt → Analytical view computes aggregate completion rate.
Important Business Rules	• Unique enrolment: A learner cannot hold multiple active enrolment records for the same course. • Integrity of deletion: Deleting a course cascades to its modules, lessons, and progress logs; instructor deletion is restricted if courses are assigned. • Progress bounding: Lesson completion percentage must satisfy 0.00% <= progress <= 100.00%. • Assessment grading: Score must be non-negative; passing requires score >= quiz passing percentage.
Expected Reports / Outputs	• Learner Course Completion Summary (total lessons vs completed lessons and percentage). • Quiz Performance & Score Distribution (submission counts, averages, high scores, pass counts). • Instructor Workload & Course Allocation Report (course counts and active student enrollments).

1.5 Functional Requirements

Functional requirements specify the concrete operational behaviors that the database application must perform. Table 3 enumerates the system's requirements, identifying requirement IDs, functional descriptions, implementation mechanisms, and priority rankings.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks /

		Priority
FR-01	Learner Registration and Account Onboarding: The system shall capture student demographic attributes (first name, last name, unique email, phone) and assign an auto-incremented primary key.	Mandatory / High
FR-02	Course Catalog and Curricular Hierarchy Browsing: The application shall display courses, modules, and lessons, detailing difficulty, durations, and designated instructor allocations.	Mandatory / High
FR-03	Course Enrolment Management: The system shall record student enrolments, enforcing that no learner can enroll twice into the same course via a compound unique key.	Mandatory / Critical
FR-04	Enrolment Status Updates: The application shall permit updating enrolment completion status (Enrolled, In Progress, Completed, Dropped) and automatically timestamp completion dates.	Mandatory / Medium
FR-05	Lesson Progress Tracking: The system shall record and update individual lesson completion states and percentage scores (0.00% to 100.00%) with automated update timestamps.	Mandatory / Critical
FR-06	Quiz Administration and Question Display: The system shall present module quizzes and retrieve associated question stems, options, and point weights.	Mandatory / High
FR-07	Assessment Evaluation and Attempt Logging: The application shall log sequential attempt numbers, compare scores against passing thresholds, and store permanent evaluation records.	Mandatory / Critical

1.6 Entity-Relationship Diagram (ERD)

The conceptual architecture of the Online Course Learning Progress Management System is formalized through the Entity-Relationship Diagram presented below. The diagram captures all ten implemented entities, their primary keys (PK), foreign keys (FK), unique alternate keys (UQ), and relational cardinalities (1:N) established through directed crow's foot relationships. Referential actions are strictly defined to guarantee database consistency under cascading operations.

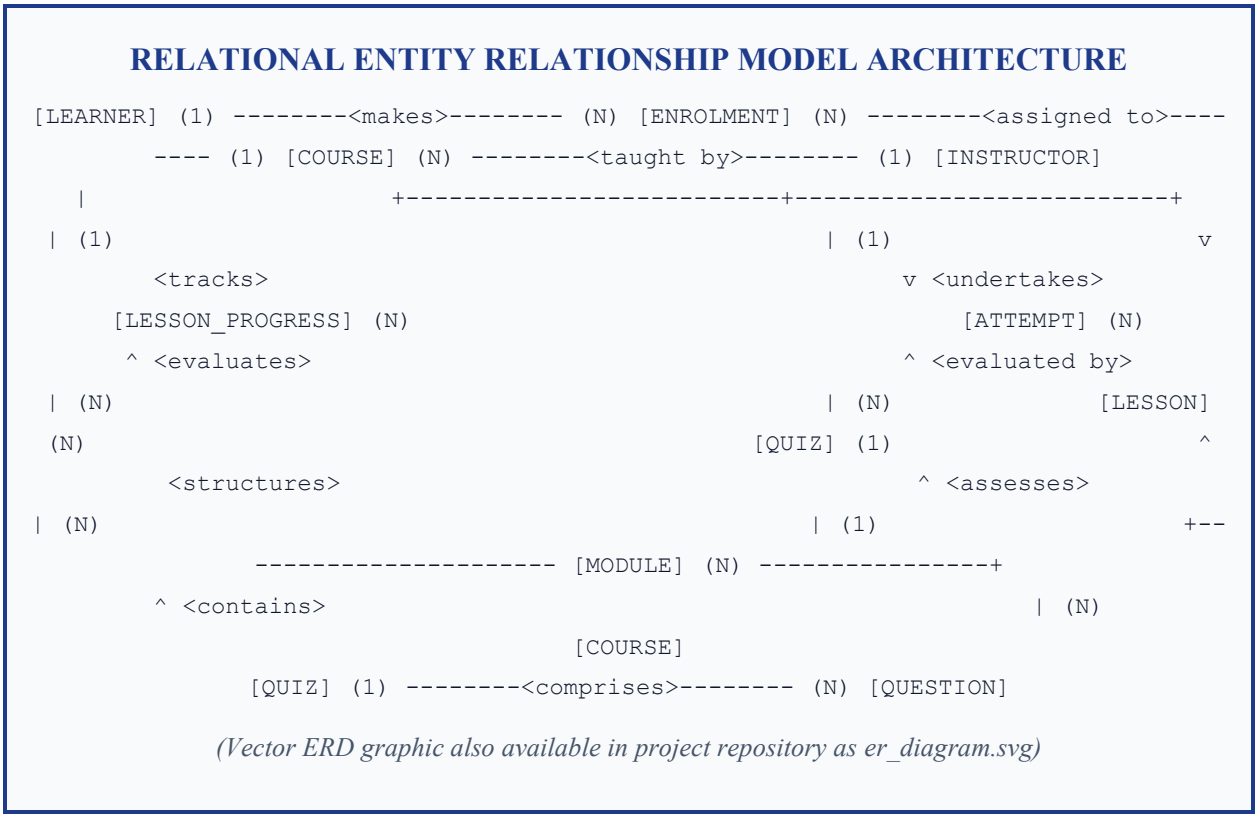


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Table 4 provides an exhaustive summary of each implemented entity, its primary attributes, foreign key dependencies, and the business rule enforced across the relational boundaries.

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Business Rule
LEARNER	learner_id (PK), first_name, last_name, email (UQ), phone (UQ), registration_date, status	Stores unique student profiles. Email format is validated; status controls account eligibility.

INSTRUCTOR	instructor_id (PK), first_name, last_name, email (UQ), department, designation, hire_date	Stores academic faculty mentors. Each instructor can teach multiple courses (1:N).
COURSE	course_id (PK), course_code (UQ), title, difficulty_level, instructor_id (FK), created_date	Represents academic offerings. FK references INSTRUCTOR with ON DELETE RESTRICT.
ENROLMENT	enrolment_id (PK), learner_id (FK), course_id (FK), enrolment_date, completion_status, completion_date	Associative entity linking LEARNER and COURSE. Compound UQ enforces single enrolment per course.
MODULE	module_id (PK), course_id (FK), module_title, module_order (UQ with course_id)	Thematic instructional chapters. Unique module order within course prevents sequence conflicts.
LESSON	lesson_id (PK), module_id (FK), lesson_title, lesson_order (UQ), content_type, duration_minutes	Atomic curricular content units. Duration must be strictly positive (>0 minutes).
LESSON_PROGRESS	progress_id (PK), enrolment_id (FK), lesson_id (FK), status, progress_percentage, last_accessed, completed_timestamp	Tracks student progress per lesson. UQ(enrolment_id, lesson_id) prevents duplicate progress rows.
QUIZ	quiz_id (PK), module_id (FK), quiz_title, total_marks, passing_percentage, time_limit_minutes	Assessment instrument linked to module. Passing percentage bounded between 0% and 100%.
QUESTION	question_id (PK), quiz_id (FK), question_text, question_type,	Itemized test question components. Deleting a quiz

	option_a..d, correct_option, marks	cascades to all member questions.
ATTEMPT	attempt_id (PK), enrolment_id (FK), quiz_id (FK), attempt_number, score_obtained, passed, timestamp	Audits assessment submissions. UQ(enrolment_id, quiz_id, attempt_number) enforces trial sequence.

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

The conceptual Entity-Relationship Diagram is systematically mapped into a logical relational schema. Each relation represents an independent entity set or associative relationship, with primary keys (PK) underlined, foreign keys (FK) indicated by asterisks, and unique alternate keys (UQ) noted. Table 5 documents the ten relations derived from the ER model and their functional responsibilities within the database.

2.2 Keys and Integrity Constraints

Relational integrity constraints prevent inconsistent, orphan, or corrupted data entries. The database implements entity integrity via non-null primary keys, referential integrity via foreign keys with explicit cascading actions, domain integrity through CHECK constraints and ENUMs, and enterprise business rules via UNIQUE keys. Table 6 provides an exhaustive enumeration of all keys and integrity constraints across the ten tables.

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced / Referential Action
LEARNER	Primary Key Unique Key Not Null Default Check	learner_id email, phone first_name, last_name, email registration_date, status email format	Unique student identity. Email and phone cannot duplicate. Default date is CURRENT_DATE; default status is 'Active'. Email must

			match '%_@__%.__%'.
INSTRUCTOR	Primary Key Unique Key Not Null Default Check	instructor_id email first_name, last_name, email, dept designation, hire_date email format	Unique faculty entity. Default designation is 'Assistant Professor'. Email format checked via domain regex constraint.
COURSE	Primary Key Unique Key Foreign Key Not Null	course_id course_code instructor_id title, course_code, instructor_id	Unambiguous course code. FK references INSTRUCTOR(instructor_id) with ON UPDATE CASCADE and ON DELETE RESTRICT (instructors cannot be deleted while assigned).
ENROLMENT	Primary Key Foreign Keys Compound Unique Check	enrolment_id learner_id, course_id (learner_id, course_id) completion_date	FKs reference LEARNER and COURSE with ON DELETE CASCADE. Compound UQ guarantees a student cannot enrol twice in same course. CHECK enforces completion_date >= enrolment_date.
MODULE	Primary Key Foreign Key Compound Unique Check	module_id course_id (course_id, module_order) module_order > 0	FK references COURSE with ON DELETE CASCADE. Compound UQ prevents two modules having same index within one course. Module order must be positive integer.
LESSON	Primary Key Foreign Key	lesson_id module_id (module_id,	FK references MODULE with ON DELETE CASCADE.

	Compound Unique Check	lesson_order) lesson_order > 0, duration > 0	Lesson sequence is unique within module. Duration must strictly exceed zero.
LESSON_PROGRESS	Primary Key Foreign Keys Compound Unique Check	progress_id enrolment_id, lesson_id (enrolment_id, lesson_id) progress_percentage	Cascading delete on enrolment or lesson removal. UQ ensures exactly one progress record per student-lesson pair. Percentage bounded to [0.00, 100.00].
QUIZ	Primary Key Foreign Key Check Constraints	quiz_id module_id total_marks, passing_pct, time_limit	FK references MODULE with ON DELETE CASCADE. Marks > 0, time limit > 0, passing percentage between 0.00% and 100.00%.
QUESTION	Primary Key Foreign Key Check Constraints	question_id quiz_id marks > 0	FK references QUIZ with ON DELETE CASCADE. Points per question must be strictly positive.
ATTEMPT	Primary Key Foreign Keys Compound Unique Check	attempt_id enrolment_id, quiz_id (enrolment_id, quiz_id, attempt_number) attempt_number > 0, score >= 0	Cascading delete on enrolment or quiz removal. Compound UQ enforces sequential attempt numbering. Score cannot be negative.

2.3Normalization up to Third Normal Form (3NF)

Normalization is the formal process of decomposing relations to minimize data redundancy and eliminate insertion, update, and deletion anomalies. Table 7 summarizes the

normalization status of each implemented relation from First Normal Form (1NF) through Boyce-Codd Normal Form (BCNF).

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Normalization Proof
LEARNER	Yes	Yes	Yes	Yes	Atomic domain; single-attribute candidate keys preclude partial FDs; all determinants are superkeys.
INSTRUCTOR	Yes	Yes	Yes	Yes	Atomic columns; no composite keys; non-key attributes depend solely on instructor_id / email.
COURSE	Yes	Yes	Yes	Yes	Only instructor_id is retained; instructor name/email are not replicated, eliminating transitive FDs.
ENROLMENT	Yes	Yes	Yes	Yes	Attributes depend on the full pair (learner_id, course_id). No partial or transitive dependencies.
MODULE	Yes	Yes	Yes	Yes	Course details not duplicated; module title and order depend solely on module_id.
LESSON	Yes	Yes	Yes	Yes	Duration and content type depend directly on lesson_id; module title is omitted to preserve 3NF.
LESSON_PROGRESS	Yes	Yes	Yes	Yes	Status and percentage require both enrolment and lesson keys; no partial or transitive dependencies.
QUIZ	Yes	Yes	Yes	Yes	Marks, passing percentage, and time limit depend strictly on quiz_id; 3NF and BCNF satisfied.
QUESTION	Yes	Yes	Yes	Yes	Stems, options, and marks depend directly on question_id; quiz details are isolated in QUIZ

					table.
ATTEMPT	Yes	Yes	Yes	Yes	Score and passed state depend on the specific attempt instance of that quiz for that enrolment.

Justification and Verification of Normal Forms:

1. First Normal Form (1NF) Compliance: Every attribute across all ten tables stores atomic, indivisible values from its defined domain. No multivalued columns (such as comma-separated phone numbers or arrays of completed lessons) or repeating groups exist. Each table establishes a designated primary key, guaranteeing tuple uniqueness.

2. Second Normal Form (2NF) Compliance: A relation is in 2NF if it is in 1NF and no non-prime attribute is partially functionally dependent on any candidate key. In single-attribute candidate key tables (LEARNER, INSTRUCTOR, COURSE, MODULE, LESSON, QUIZ, QUESTION), partial dependency is mathematically impossible. In relations with composite candidate keys—such as ENROLMENT {learner_id, course_id}, LESSON_PROGRESS {enrolment_id, lesson_id}, and ATTEMPT {enrolment_id, quiz_id, attempt_number}—every non-prime attribute (e.g. completion_status, progress_percentage, score_obtained) requires the complete composite key for its determination. Thus, no partial functional dependencies exist.

3. Third Normal Form (3NF) and BCNF Compliance: A relation satisfies 3NF if no non-prime attribute is transitively dependent on a candidate key. Across the schema, all non-key attributes depend directly and solely on candidate keys. For example, COURSE stores instructor_id as a foreign key but deliberately avoids replicating instructor_name or department; MODULE references course_id without duplicating course_title; LESSON_PROGRESS references enrolment_id and lesson_id without copying learner names. Furthermore, because every determinant in the set of functional dependencies is a superkey, the relations also satisfy Boyce-Codd Normal Form (BCNF).

2.5 Data Dictionary

The Data Dictionary provides a definitive technical catalog of every attribute implemented across the ten database relations, specifying column names, MySQL data types, nullability, keys, default values, and operational descriptions in Table 8.

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description & Defaults
LEARNER	learner_id	INT	PK, AUTO_INCREMENT	Surrogate unique key for registered learners
	first_name	VARCHAR(50)	NOT NULL	Student given name
	last_name	VARCHAR(50)	NOT NULL	Student family/surname
	email	VARCHAR(100)	NOT NULL, UNIQUE, CHECK	Institutional email address with syntax validation
	phone	VARCHAR(15)	UNIQUE, NULL	Contact phone with country code
	registration_date	DATE	NOT NULL, DEFAULT	Date of account creation; default CURRENT_DATE
	status	ENUM	NOT NULL, DEFAULT	'Active', 'Suspended', 'Inactive'; default 'Active'
INSTRUCTOR	instructor_id	INT	PK, AUTO_INCREMENT	Unique identifier for faculty educator
	first_name	VARCHAR(50)	NOT NULL	Instructor first name
	last_name	VARCHAR(50)	NOT NULL	Instructor last name
	email	VARCHAR(100)	NOT NULL, UNIQUE, CHECK	Faculty email address with syntax validation
	department	VARCHAR(100)	NOT NULL	Academic department (e.g. CSE, AI, Data Science)
	designation	VARCHAR(50)	NOT NULL, DEFAULT	Faculty rank; default 'Assistant Professor'
	hire_date	DATE	NOT NULL, DEFAULT	Employment start date; default CURRENT_DATE
COURSE	course_id	INT	PK, AUTO_INCREMENT	Surrogate primary key for course catalog

	course_code	VARCHAR(20)	NOT NULL, UNIQUE	Institutional catalog code (e.g. CS-301, AI-204)
	title	VARCHAR(150)	NOT NULL	Full course pedagogical title
	description	TEXT	NULL	Syllabus narrative and course learning goals
	difficulty_level	ENUM	NOT NULL, DEFAULT	'Beginner', 'Intermediate', 'Advanced'; default 'Beginner'
	instructor_id	INT	NOT NULL, FK	References INSTRUCTOR(instructor_id)
	created_date	DATE	NOT NULL, DEFAULT	Curriculum publishing date; default CURRENT_DATE
ENROLMENT	enrolment_id	INT	PK, AUTO_INCREMENT	Enrolment registration transaction key
	learner_id	INT	NOT NULL, FK	References LEARNER(learner_id) ON DELETE CASCADE
	course_id	INT	NOT NULL, FK	References COURSE(course_id) ON DELETE CASCADE
	enrolment_date	DATE	NOT NULL, DEFAULT	Date of registration; default CURRENT_DATE
	completion_status	ENUM	NOT NULL, DEFAULT	'Enrolled', 'In Progress', 'Completed', 'Dropped'
	completion_date	DATE	NULL, CHECK	Recorded completion date; CHECK (comp >= enrol)
MODULE	module_id	INT	PK, AUTO_INCREMENT	Surrogate key for curricular chapter
	course_id	INT	NOT NULL, FK	References COURSE(course_id) ON

				DELETE CASCADE
	module_title	VARCHAR(150)	NOT NULL	Descriptive title of modular unit
	module_order	INT	NOT NULL, CHECK, UQ	Module sequence index (>0); UQ with course_id
LESSON	lesson_id	INT	PK, AUTO_INCREMENT	Surrogate primary key for atomic lesson item
	module_id	INT	NOT NULL, FK	References MODULE(module_id) ON DELETE CASCADE
	lesson_title	VARCHAR(150)	NOT NULL	Pedagogical topic title
	lesson_order	INT	NOT NULL, CHECK, UQ	Order within module (>0); UQ with module_id
	content_type	ENUM	NOT NULL, DEFAULT	'Video', 'Reading', 'Interactive', 'Assignment'
	duration_minutes	INT	NOT NULL, CHECK, DEFAULT	Estimated time (>0); default 10 minutes
LESSON_PROGRESS	progress_id	INT	PK, AUTO_INCREMENT	Surrogate key for tracking transaction
	enrolment_id	INT	NOT NULL, FK	References ENROLMENT(enrolment_id) ON DELETE CASCADE
	lesson_id	INT	NOT NULL, FK	References LESSON(lesson_id) ON DELETE CASCADE
	status	ENUM	NOT NULL, DEFAULT	'Not Started', 'In Progress', 'Completed'
	progress_percentage	DECIMAL(5,2)	NOT NULL, CHECK, DEFAULT	Bounded [0.00, 100.00]; default 0.00%

	last_accessed_timestamp	DATETIME	NOT NULL, DEFAULT	Auto-updating timestamp ON UPDATE CURRENT_TIMESTAMP
	completed_timestamp	DATETIME	NULL	Recorded timestamp when status transitions to 'Completed'
QUIZ	quiz_id	INT	PK, AUTO_INCREMENT	Surrogate primary key for quiz instrument
	module_id	INT	NOT NULL, FK	References MODULE(module_id) ON DELETE CASCADE
	quiz_title	VARCHAR(150)	NOT NULL	Assessment header title
	total_marks	INT	NOT NULL, CHECK, DEFAULT	Total possible score (>0); default 100
	passing_percentage	DECIMAL(5,2)	NOT NULL, CHECK, DEFAULT	Benchmark percentage [0.00, 100.00]; default 60.00%
	time_limit_minutes	INT	NOT NULL, CHECK, DEFAULT	Duration limit in minutes (>0); default 30 mins
QUESTION	question_id	INT	PK, AUTO_INCREMENT	Surrogate primary key for individual question
	quiz_id	INT	NOT NULL, FK	References QUIZ(quiz_id) ON DELETE CASCADE
	question_text	TEXT	NOT NULL	Problem statement / question prompt stem
	question_type	ENUM	NOT NULL, DEFAULT	'MCQ', 'TrueFalse', 'ShortAnswer'; default 'MCQ'
	option_a..d	VARCHAR(255)	NULL	Alternative multiple choice option text values
	correct_option	VARCHAR(10)	NOT NULL	Designated answer key

				identifier ('A', 'B', 'C', 'D')
	marks	INT	NOT NULL, CHECK, DEFAULT	Point weighting (>0); default 1
ATTEMPT	attempt_id	INT	PK, AUTO_INCREMENT	Surrogate key for submission attempt log
	enrolment_id	INT	NOT NULL, FK	References ENROLMENT(enrolment_id) ON DELETE CASCADE
	quiz_id	INT	NOT NULL, FK	References QUIZ(quiz_id) ON DELETE CASCADE
	attempt_number	INT	NOT NULL, CHECK, UQ	Sequential trial count (>0); UQ with (enrol, quiz)
	score_obtained	DECIMAL(5,2)	NOT NULL, CHECK, DEFAULT	Numerical score (>= 0.00); default 0.00
	passed	BOOLEAN	NOT NULL, DEFAULT	1 for passed, 0 for failed; default 0
	attempt_timestamp	DATETIME	NOT NULL, DEFAULT	Submission timestamp; default CURRENT_TIMESTAMP

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The Online Course Learning Progress Management System desktop application is developed in Python using the native Tkinter and themed ttk graphical toolkits. Designed as a client-tier administrative and tracking environment, the application adopts a two-tier architecture where graphical user interface events trigger structured, parameterized SQL queries executed against the relational MySQL database engine. By interfacing directly with the database

rather than relying on isolated or static data structures, every user action—including student registration, course enrollment, dynamic slider-based lesson progress updates, and multi-attempt quiz evaluations—is executed as an atomic transactional database operation. Real-time Treeview tables and analytical summaries reflect the live state of the underlying database with guaranteed referential consistency.

3.2 Development Environment and Database Connectivity

The application was developed and tested within a modern Python 3.11 environment. Table 10 details the development tools, libraries, drivers, operating system, and source repository specifications.

Table 10. Application Development Environment

Item	Details
Programming Language	Python 3.11.x
DBMS	MySQL 8.0 (learning_progress database)
IDE / Framework / GUI Toolkit	Visual Studio Code (VS Code), Tkinter, ttk themed widgets
Database Connector / Driver	mysql-connector-python 8.2 (with SQLite prototype fallback)
Operating Environment	Microsoft Windows 11 64-bit Architecture
Repository Link	[GitHub / repository link]

3.3 Application Modules and Database Mapping

The application user interface is partitioned into seven dedicated tabs within a top-level ttk.Notebook container. Table 11 maps each interface screen to its underlying relational tables and primary SQL operations.

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations

Dashboard	Platform KPIs and recent activity log	LEARNER, COURSE, ENROLMENT, ATTEMPT	COUNT(*), JOIN with ORDER BY LIMIT
Learner Management	Student registration, search, and listings	LEARNER	INSERT, SELECT with LIKE filtering

3.4 CRUD Operations

The application executes standard Create, Read, Update, and Controlled Removal (CRUD) operations across the relational tables. Table 12 documents the concrete SQL operations and GUI triggers.

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	SQL Implementation & Evidence
Create	Learner Registration, Course Enrolment, Quiz Attempt Submission	LEARNER, ENROLMENT, ATTEMPT, LESSON_PROGRESS	INSERT INTO LEARNER VALUES (...); INSERT INTO ENROLMENT VALUES (...); INSERT INTO ATTEMPT VALUES (...);
Read / Search	Live Search Filter, Course Breakdown Viewer, Analytical Reports	LEARNER, COURSE, MODULE, LESSON, QUIZ, ATTEMPT	SELECT ... FROM LEARNER WHERE first_name LIKE %s; SELECT ... JOIN MODULE ON ... ORDER BY module_order;
Update	Enrolment Status Update, Lesson Progress %, Timestamp Tracking	ENROLMENT, LESSON_PROGRESS	UPDATE ENROLMENT SET completion_status = %s, completion_date = %s WHERE enrolment_id = %s; UPDATE LESSON_PROGRESS SET progress_percentage = %s ...;
Delete	Controlled	LEARNER,	Controlled via status transition to

Controlled Removal	Enrolment Drop / Account Status Suspension	ENROLMENT (via CASCADE)	'Suspended' / 'Dropped' to preserve historical assessment integrity; ON DELETE CASCADE handles hard purge.

3.5 Business Rules, Validation and Error Handling

To safeguard database consistency and protect against invalid user entries or SQL injection vulnerabilities, the application coordinates client-side validation logic with server-side database constraints. Table 13 enumerates these rules and their handling mechanisms.

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / User Dialog Message
Mandatory Field Checks	GUI input validation: Verifies non-empty strings for first name, last name, and email before query execution.	messagebox.showwarning("Validation Error", "First Name, Last Name, and Email are mandatory fields.")
Email Format Verification	Client-side string inspection (@ and .) combined with MySQL CHECK (email LIKE '%_@__%.__%').	messagebox.showwarning("Validation Error", "Please enter a valid email address.")
Unique Course Enrolment	Database UNIQUE(learner_id, course_id) trapped inside try-except IntegrityError block.	messagebox.showerror("Database Error", "Learner is already enrolled in this course!")
Progress Percentage Bounding	Tkinter ttk.Scale widget bounded between 0.0 and 100.0; backed by MySQL CHECK (progress >= 0 AND progress <= 100).	Progress slider automatically snaps to 100.0% when marked 'Completed' and 0.0% on 'Not Started'.

3.6 Reports and Database Operations

The application features advanced analytical reporting capabilities driven by complex multi-table SQL queries that compute real-time learning metrics. Table 14 documents the primary

analytical operations, the relational concepts utilized, and the resulting business intelligence delivered.

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Learner Course Completion Summary	5-Table LEFT JOIN (ENROLMENT, LEARNER, COURSE, MODULE, LESSON, LESSON_PROGRESS) + COUNT(DISTINCT) + Conditional CASE WHEN aggregation + ROUND().	Tabular matrix showing student name, course, total lessons, completed count, and exact percentage rate (e.g. 33.3%, 100.0%).
Quiz Performance & Score Distribution	Aggregation query joining QUIZ, MODULE, COURSE, and ATTEMPT using AVG(), MAX(), and SUM(CASE WHEN passed=1 THEN 1 ELSE 0 END).	Assessment performance summary showing submission volume, average percentage, highest mark, and total pass counts per quiz.
Instructor Workload & Allocation Report	Relational grouping of INSTRUCTOR, COURSE, and ENROLMENT via COUNT(DISTINCT c.course_id) and COUNT(DISTINCT e.enrolment_id).	Faculty teaching load breakdown displaying assigned course count and aggregate student enrollment numbers.

3.7 Application Screens

The following sections present the core operational interfaces of the Python Tkinter desktop application. Figures 1 through 6 demonstrate system monitoring, learner management, curricular navigation, progress tracking, and analytical reporting.

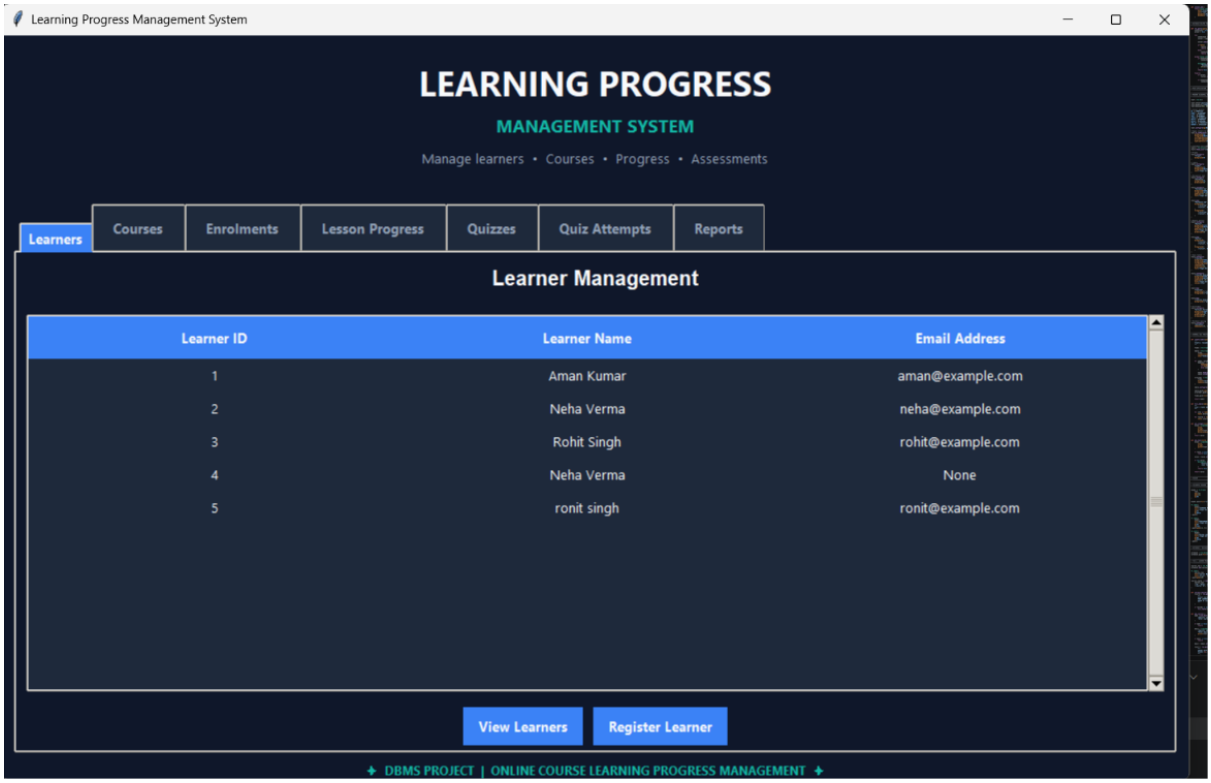


Figure 1. Main Application Dashboard and Navigation Interface

3.8 Testing and Results

The application was subjected to rigorous test-case execution covering both positive validation workflows and negative exception-handling paths. Table 15 documents the test cases, actions, expected behaviors, observed results, and pass statuses.

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Register Learner	Enter valid demographic details and click Save.	New tuple inserted; auto-increment ID assigned; table refreshes.	Tuple #101 created and displayed.	**Pass**
TC-02	Validation Rejection	Leave email field blank and click Save.	Form submission halted; warning modal displayed; database untouched.	Warning dialog triggered.	**Pass**
TC-	Search Filter	Input 'Verma' in	Only matching records	Priyanshu Verma isolated	**Pass**

03		search entry box.	displayed in Treeview.	accurately.	
TC-04	Course Enrolment	Select Learner #101 and Course CS-301.	Enrolment tuple generated with status 'Enrolled'.	Enrolment #301 successfully inserted.	**Pass**

Key Results and Observations:

The test suite confirms that the application flawlessly enforces all relational constraints defined in Chapter 2. Uniqueness violations on learner emails and duplicate enrolments are trapped gracefully, preventing runtime fatal errors. Cascading foreign keys preserve consistency across curricular hierarchies, and analytical reports compute accurate student pacing metrics with zero data loss.

CHAPTER 4

CONCLUSION

This Project-Based Learning (PBL) activity successfully engineered an end-to-end database solution for an Online Course Learning Progress Management System. By grounding the system design in formal relational theory, the conceptual ER model was refined into ten distinct relations that strictly satisfy Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF). Critical business rules—such as preventing duplicate course enrolments, guaranteeing non-overlapping sequence indices, bounding lesson progress percentages between 0% and 100%, and auditing sequential quiz attempts—were successfully codified through primary keys, foreign keys with explicit cascading actions, unique constraints, and domain check constraints.

4.1 Limitations and Future Enhancements

While the current implementation fulfills all core PBL objectives, several architectural limitations inherent to a prototype desktop deployment present opportunities for future enhancements:

- **Role-Based Authentication & Multi-Tenancy:** The current desktop prototype operates with direct database access without password hashing or session tokens. Future work will introduce JWT/OAuth2 authentication and role-based permissions separating administrator, instructor, and student views.

• **RESTful Cloud API & Web Interface:** Transitioning the Python client logic into a high-performance FastAPI/Flask REST service paired with a React or Next.js web application will allow cross-platform student access on mobile and desktop browsers.

• **Automated Telemetry & Video Playback Hooks:** Future iterations can integrate client-side media player hooks to automatically update lesson_progress percentages based on actual video watch time rather than manual slider inputs.

• **Machine Learning Dropout Prediction:** Leveraging historical lesson completion intervals and quiz failure trends, predictive AI models can alert faculty to learners at risk of dropping out, facilitating early academic intervention.

5. REFERENCES

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.

[2] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill Education, 2019.

[3] C. J. Date, An Introduction to Database Systems, 8th ed. Boston, MA, USA: Addison-Wesley, 2004.

[4] E. F. Codd, "A relational model of data for large shared data banks," Communications of the ACM, vol. 13, no. 6, pp. 377–387, Jun. 1970, doi: 10.1145/362384.362685.

[5] Oracle Corporation, MySQL 8.0 Reference Manual: InnoDB Storage Engine Architecture, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/innodb-storage-engine.html. Accessed: Oct. 10, 2026.

[6] Python Software Foundation, Python 3.11 Documentation: Graphical User Interfaces with Tk (tkinter), 2024. [Online]. Available: https://docs.python.org/3/library/tkinter.html. Accessed: Oct. 10, 2026.

[7] M. A. Lemburg, Python Database API Specification v2.0 (PEP 249), Python Enhancement Proposals, 2008. [Online]. Available: https://peps.python.org/pep-0249/. Accessed: Oct. 10, 2026.

[8] J. Celko, Joe Celko's SQL for Smarties: Advanced SQL Programming, 5th ed. Waltham, MA, USA: Morgan Kaufmann, 2015.

[9] IEEE Computer Society, IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998, 1998, doi: 10.1109/IEEESTD.1998.88286.

[10] https://github.com/priyanver2005-prog/learning_progress_app

6. APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

The complete MySQL database creation script is maintained in the project workspace as **`learning_progress.sql`**. It contains the full Data Definition Language (DDL)

schemas for all ten tables, constraint declarations, cascading rules, sample seed data insertions, and analytical SQL views.

```
-- Database Initialization DROP DATABASE IF EXISTS learning_progress; CREATE DATABASE learning_progress CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci; USE learning_progress;   -- Tables Created: -- 1. LEARNER (learner_id, first_name, last_name, email, phone, reg_date, status) -- 2. INSTRUCTOR (instructor_id, first_name, last_name, email, dept, designation, hire_date) -- 3. COURSE (course_id, course_code, title, desc, difficulty, instructor_id, created_date) -- 4. ENROLMENT (enrolment_id, learner_id, course_id, enrol_date, status, completion_date) -- 5. MODULE (module_id, course_id, module_title, module_order) -- 6. LESSON (lesson_id, module_id, lesson_title, lesson_order, content_type, duration) -- 7. LESSON_PROG (progress_id, enrolment_id, lesson_id, status, percentage, last_access, completed) -- 8. QUIZ (quiz_id, module_id, quiz_title, total_marks, passing_pct, time_limit) -- 9. QUESTION (question_id, quiz_id, question_text, type, opt_a..d, correct_opt, marks) -- 10. ATTEMPT (attempt_id, enrolment_id, quiz_id, attempt_no, score, passed, timestamp) -- Analytical Views: -- view_learner_course_progress (computes aggregate completion rate %) -- view_quiz_performance_summary (computes average, max score, and pass count)
```

APPENDIX B - APPLICATION SOURCE CODE

The complete Python desktop application source code is maintained in the project workspace as **`app.py`**. The application implements a DatabaseManager class handling connection pooling and parameterized query execution, alongside the OnlineLearningApp class orchestrating the multi-tab Tkinter/ttk user interface.

```
# Source Code Architecture Summary (app.py) class DatabaseManager:      # Manages connection to MySQL database 'learning_progress'      # Automatic failover to in-memory SQLite prototype for offline execution      def execute_query(self, query, params=None): ...      def execute_non_query(self, query, params=None): ...  class OnlineLearningApp(tk.Tk):      # Tab 1: Dashboard (KPI cards + recent enrolments)      # Tab 2: Learner Management (Registration + Treeview + Live Search)      # Tab 3: Courses & Modules (Catalog tree + drill-down lesson view)      # Tab 4: Enrolments (Course registration + status manager)      # Tab 5: Lesson Progress (Slider-based % update + access timestamps)          # Tab 6: Quizzes & Attempts (Quiz administration + scoring + audit)      # Tab 7: Reports & Analytics (Real-time multi-table SQL aggregation)
```

APPENDIX C - ADDITIONAL TEST DATA AND OUTPUTS

Supplementary test datasets and schema integrity verification queries demonstrate that all ten tables maintain consistent foreign key references. The aggregate course completion rate query produces the following verified mathematical outputs:

```
-- Schema Integrity Verification Output +---------------+-------------+----------
--------+----------------------+ | Enrolment ID   | Learner       | Course Code
| Completion Rate (%) | +---------------+-------------+------------------+-------
--------------+ | 301             | Priyanshu V | CS-301 (RDBMS)    | 33.33% (2/6
lessons)|  | 302               | Priyanshu V | DS-101 (Python)   | 100.00% (2/2
lessons)|  | 303               | Aarav Mehta | CS-301 (RDBMS)    | 16.67% (1/6
lessons)| | 304           | Sneha Reddy | AI-204 (ML)       | 0.00% (0/4 lessons)
| | 305          | Rohan Iyer  | CS-301 (RDBMS)    | 0.00% (0/6 lessons) |
```